

Image Processing Project - Beginner's Guide

■ What is This Project?

This is an **Image Processing Desktop Application** built with Python. Think of it as a photo editor that can: - Change how images look (make them grayscale, blur them, etc.) - Transform images (rotate, resize, crop) - Analyze images (show histograms, detect edges) - Compress images (make files smaller using different techniques)

Important: All the image processing operations are built from scratch using basic math operations - no shortcuts!

■ Project Structure (How Files Are Organized)

```
src/
■■■ main.py           # Starting point - launches the app
■■■ gui.py            # The user interface (buttons, windows, displays)
■■■ image_loader.py   # Handles loading/saving images
■■■ utils.py          # Helper functions used by other modules
■
■■■ grayscale.py      # Converts color images to black & white
■■■ binary.py         # Converts images to pure black/white
■■■ affine.py         # Transformations (rotate, scale, etc.)
■■■ interpolation.py   # Resizing images (nearest, bilinear, bicubic)
■■■ histogram.py      # Image analysis and equalization
■■■ filters.py        # Blurring filters (Gaussian, Median)
■■■ highpass.py       # Edge detection filters
■
■■■ compression/      # Folder with 10 compression algorithms
    ■■■ huffman.py
    ■■■ lzw.py
    ■■■ arithmetic.py
    ■■■ ... (7 more)
```

■■ Understanding Image Processing Basics

What is an Image (Digitally)?

An image is a **grid of pixels** (tiny colored squares): - Each pixel has a color (Red, Green, Blue values from 0-255) - A 100×100 image = 10,000 pixels - Color images have 3 values per pixel (RGB) - Grayscale images have 1 value per pixel (brightness 0-255)

Example: A pixel with RGB(255, 0, 0) = bright red - R=255 (maximum red) - G=0 (no green) - B=0 (no blue)

■ Core Features Explained (For Beginners)

1. Image Reading & Display

What it does: Loads an image file and shows information about it.

How it works: - You click "Upload Image" - The program reads the file - It displays: width × height, file size, and image type

Why it's important: You need to know what you're working with before processing!

2. Grayscale Conversion

What it does: Converts a color image to black & white (grayscale).

How it works: - Takes each pixel's RGB values - Uses a formula: $\text{Gray} = 0.299 \times R + 0.587 \times G + 0.114 \times B$ - These weights (0.299, 0.587, 0.114) match how human eyes perceive brightness - Result: One brightness value per pixel (0 = black, 255 = white)

Real-world example: Like converting a color photo to black & white in old cameras.

3. Binary Image Conversion

What it does: Converts grayscale to pure black/white (no gray shades).

How it works: 1. Calculates **threshold** = average brightness of all pixels 2. If pixel brightness $\geq$ threshold $\rightarrow$ White (255) 3. If pixel brightness $<$ threshold $\rightarrow$ Black (0)

Threshold Evaluation: - Checks if the threshold is "good" by seeing if it splits dark/bright regions evenly - If too many pixels become black $\rightarrow$ threshold too high - If too many pixels become white $\rightarrow$ threshold too low

Real-world example: Like a photocopier that only prints black or white, no gray.

4. Affine Transformations

What it does: Changes the position, size, or shape of an image.

Types:

a) Translation (Moving)

- Moves image left/right, up/down
- Like sliding a picture on a table

b) Scaling (Resizing)

- Makes image bigger or smaller
- Can stretch width/height independently

c) Rotation (Turning)

- Rotates image by an angle (e.g., 30° , 90°)
- Like spinning a photo

d) Shear (Skewing)

- Tilts the image horizontally or vertically
- Like leaning a picture

How it works: Uses **matrices** (math grids) to calculate new pixel positions.

5. Image Interpolation (Resizing)

What it does: Changes image size while trying to keep it looking good.

Three Methods:

a) Nearest Neighbor

- Simplest method
- Finds closest pixel from original
- Fast but can look blocky/pixelated

b) Bilinear

- Smoother than nearest neighbor
- Averages 4 nearby pixels
- Good balance of speed and quality

c) Bicubic

- Best quality
- Uses 16 nearby pixels with smooth curves
- Slower but looks best

Real-world example: Like zooming in on a photo - better methods keep it sharp.

6. Cropping

What it does: Cuts out a rectangular portion of the image.

How to use: - Method 1: Enter coordinates (x1, y1, x2, y2) - Method 2: Click and drag on the preview canvas

Real-world example: Like cutting a photo with scissors to keep only part of it.

7. Histogram Analysis

What is a Histogram? A graph showing how many pixels have each brightness level (0-255).

What it tells you: - **Good histogram:** Spread across all brightness levels = good contrast - **Narrow histogram:** Most pixels similar brightness = low contrast (dull image) - **Too many dark pixels:** Image is underexposed (too dark) - **Too many bright pixels:** Image is overexposed (too bright)

Histogram Equalization: - Spreads out the histogram to use full brightness range - Improves contrast automatically - Makes dark images brighter and more visible

Real-world example: Like adjusting brightness/contrast on your TV to see details better.

8. Low-Pass Filtering (Blurring)

What it does: Smooths/blurs images to reduce noise or details.

a) Gaussian Filter (19×19, $\sigma=3$)

- Creates a blur effect
- Uses a bell-shaped curve (Gaussian) to average pixels

- 19×19 = checks 361 nearby pixels
- $\sigma=3$ controls how strong the blur is

b) Median Filter (7×7)

- Removes "salt and pepper" noise (random black/white dots)
- Replaces each pixel with the median (middle value) of nearby pixels
- 7×7 = checks 49 nearby pixels

Real-world example: Like putting vaseline on a camera lens (Gaussian) or removing dust spots (Median).

9. High-Pass Filtering (Edge Detection)

What it does: Finds edges and sharp changes in the image.

a) Laplacian Filter

- Detects edges using second derivatives (rate of change of rate of change)
- Shows where brightness changes rapidly
- Good for finding all edges

b) Sobel Filter

- Detects edges in horizontal and vertical directions
- Combines both to show edge strength
- Very popular for edge detection

c) Gradient Filter

- Uses first derivatives (rate of change)
- Shows where brightness changes
- Simpler than Sobel

Real-world example: Like tracing the outline of objects in a photo with a pen.

10. Image Compression

What it does: Makes image files smaller while trying to keep quality.

Why compress? - Smaller files = faster upload/download - Less storage space needed - But: Some quality may be lost

10 Compression Methods:

1. Huffman Coding

- Gives short codes to frequent pixel values
- Gives long codes to rare pixel values
- Like Morse code: common letters (E) = short, rare letters (Z) = long

2. Golomb-Rice Coding

- Good for images with small differences between pixels
- Divides numbers into quotient and remainder
- Efficient for certain patterns

3. Arithmetic Coding

- Very efficient compression
- Encodes entire image as one number between 0 and 1
- Can achieve near-optimal compression

4. LZW (Lempel-Ziv-Welch)

- Finds repeated patterns
- Builds a dictionary of patterns
- Replaces patterns with short codes
- Used in GIF images

5. RLE (Run-Length Encoding)

- Simple: "5 white pixels, 3 black pixels" → "5W3B"
- Works great for images with large same-color areas
- Used in simple image formats

6. Symbol-Based Coding

- Identifies most common pixel values (top 16)
- Gives them short codes
- Others get longer codes

7. Bit-Plane Coding

- Splits image into 8 layers (one per bit)
- Compresses each layer separately
- Some layers (high bits) are more important

8. DCT (Discrete Cosine Transform)

- Used in JPEG compression
- Converts 8×8 pixel blocks into frequency components
- Removes high-frequency details (less visible to humans)
- Quantizes (rounds) coefficients to save space

9. Predictive Coding (DPCM)

- Predicts next pixel based on previous pixels
- Stores only the difference (error)
- Differences are usually small → compress well

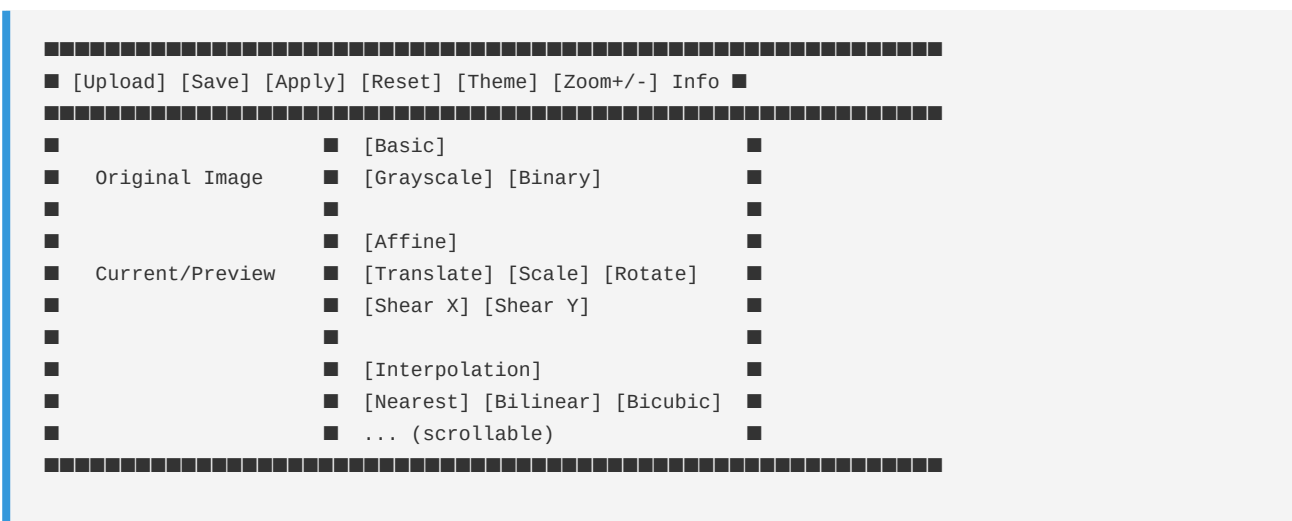
10. Wavelet Coding

- Uses wavelets (small waves) to analyze image
- Separates image into different frequency bands
- Removes small coefficients (details)
- Used in JPEG2000

Compression Metrics: - **Compression Ratio:** Original size ÷ Compressed size (higher = better) - **PSNR** (Peak Signal-to-Noise Ratio): Quality measure in decibels (higher = better quality)

■ How the GUI Works

Main Window Layout



Workflow

1. **Upload Image** → Image appears in "Original" canvas
2. **Click Operation** → Result appears in "Current/Preview" canvas
3. **Review Result** → See if you like it
4. **Apply Preview** → Makes preview the new current image
5. **Reset** → Go back to original image

Key Concepts

- **Preview Mode:** See result before committing (like "undo" safety)
 - **Dual Canvas:** Compare original vs. processed side-by-side
 - **Sections:** Related operations grouped together for easy finding
-

■ Technical Concepts (Simplified)

What is "Custom Implementation"?

Requirement: Don't use functions that do everything for you.

Example - BAD (using shortcut):

```
blurred = cv2.GaussianBlur(image, (19, 19), 3) # ■ Too easy!
```

Example - GOOD (custom implementation):

```
# 1. Create Gaussian kernel manually
kernel = create_gaussian_kernel(19, 3)

# 2. Apply convolution manually (loop through pixels)
for each pixel:
    sum = 0
    for each neighbor in 19x19 area:
        sum += neighbor_value × kernel_weight
    new_pixel = sum
```

Why? To understand HOW it works, not just use it!

What is Convolution?

Simple explanation: - Take a small grid (kernel/filter) like 3×3 - Place it over each pixel - Multiply overlapping values - Sum them up - That's the new pixel value

Example:

Original:		Kernel:		Result:
[1 2 3]		[0 -1 0]		[calculated]
[4 5 6]	×	[-1 5 -1]	=	[new pixel]
[7 8 9]		[0 -1 0]		[values]

What is Interpolation?

Problem: When resizing, new pixel positions don't match old ones.

Solution: Estimate new pixel values from nearby old pixels.

Nearest Neighbor: Use closest pixel (simple) **Bilinear:** Average 4 closest pixels (smooth) **Bicubic:** Use 16 pixels with smooth curve (best quality)

■ How to Use the Application

Step-by-Step Tutorial

Start the Application `bash py run.py`

Load an Image

3. Click "Upload Image"

4. Select a JPG, PNG, or BMP file

5. Image appears in "Original" canvas

Info shows: "800x600 | 245678 bytes | RGB"

Try Grayscale

8. Click "Grayscale" button

9. Preview shows black & white version

Click "Apply Preview" to keep it

Try Binary

12. Click "Binary" button

13. Image becomes pure black/white

Check the evaluation message (is threshold good?)

Try Rotation

16. In "Affine" section, set "Angle" to 45

17. Click "Rotate"

Image rotates 45 degrees

Try Resizing

20. In "Interpolation" section, set width=400, height=300

21. Try "Nearest" (fast, blocky)

22. Try "Bilinear" (smooth)

Try "Bicubic" (best quality, slower)

Analyze Histogram

25. Click "Compute Histogram"

26. See the graph at bottom

27. Read the evaluation (good/bad contrast?)

Click "Equalize" to improve contrast

Apply Filters

30. Try "Gaussian 19x19" (blur)

31. Try "Median 7x7" (remove noise)

Try "Sobel" (find edges)

Crop Image

34. Method 1: Enter coordinates (x1=100, y1=100, x2=300, y2=300)

35. Method 2: Click and drag on preview canvas

Click "Crop by coords" or drag to select

Test Compression

- Try "Huffman" (shows compression ratio and PSNR)
- Try "RLE" (simple, fast)
- Compare different methods

■ Learning Path (If You Want to Understand Code)

Beginner Level

1. Start with `grayscale.py` - simple math operations
2. Look at `binary.py` - basic threshold logic
3. Check `image_loader.py` - file I/O basics

Intermediate Level

1. Study `interpolation.py` - understand nearest neighbor first
2. Look at `filters.py` - see how convolution works
3. Examine `histogram.py` - array operations

Advanced Level

1. Dive into `affine.py` - matrix math
2. Study compression algorithms - data structures and algorithms
3. Explore `gui.py` - event-driven programming

■ Common Questions

Q: Why is some operations slow?

A: Custom implementations use loops instead of optimized libraries. Large images + big filters (19×19) = many calculations.

Q: What if an operation doesn't work?

A: Make sure you've loaded an image first. Some operations need grayscale - the app converts automatically.

Q: Can I undo changes?

A: Use "Reset" to go back to original. "Apply Preview" commits changes - use carefully!

Q: What image formats work?

A: PNG, JPG, JPEG, BMP. The app converts everything to RGB internally.

Q: Why are compression ratios different?

A: Different algorithms work better for different image types. RLE works great for simple images, DCT for photos.

■ Key Takeaways

1. **Images are just numbers** - arrays of pixel values
2. **Processing = math operations** - averaging, multiplying, comparing
3. **Custom implementation** - build from scratch to understand
4. **Preview before commit** - safety feature to try things
5. **Different methods** - trade-offs between speed, quality, and complexity

■ Project Requirements Summary

- **Custom Implementation:** All operations built from basic math
 - **Aesthetic GUI:** Organized, themed, user-friendly
 - **All Features:** Every required operation implemented
 - **Creative Extras:** Preview mode, theme toggle, zoom, dual canvas
-

■ Next Steps

1. **Run the application** and try each feature
2. **Read the code** starting with simpler modules
3. **Experiment** with different images and parameters
4. **Modify code** to see what happens (make backups first!)
5. **Learn the math** behind each operation for deeper understanding

Remember: This project demonstrates that complex image processing is just organized math operations on pixel arrays. Understanding the basics helps you appreciate how photo editing software works!